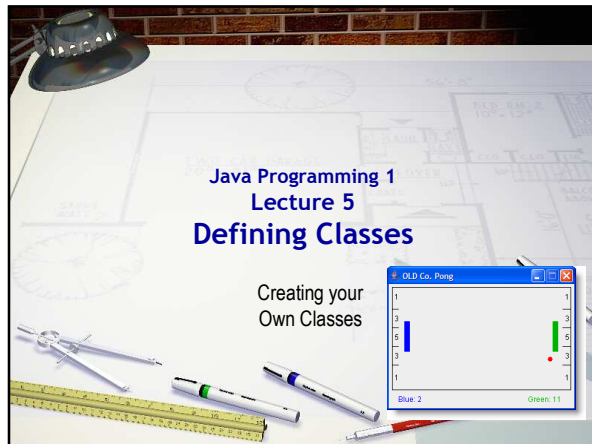
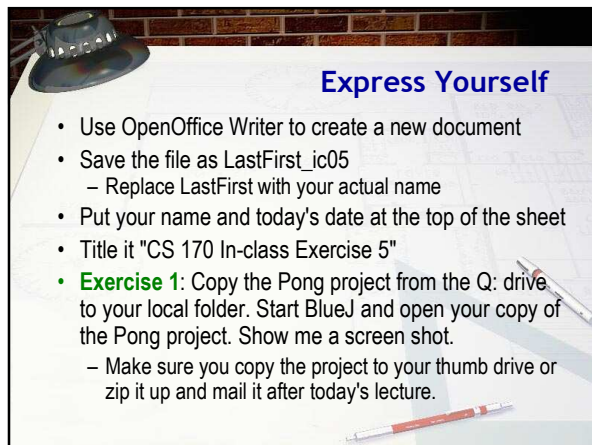
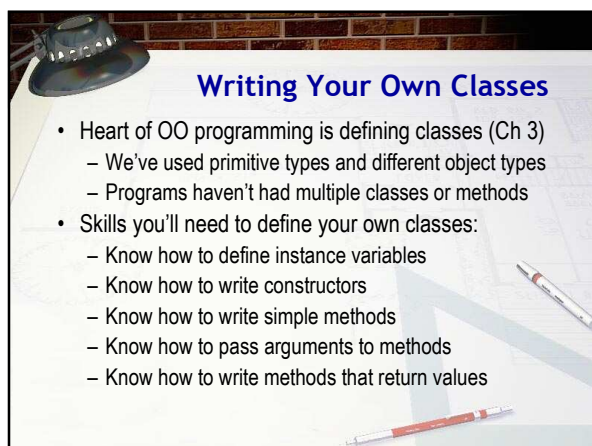


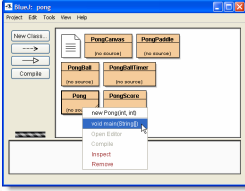






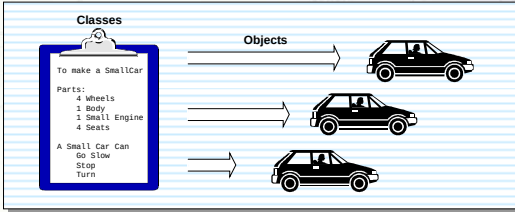
Getting Setup

- Notice that Pong classes don't have source with them
- Run the Pong game by right-clicking
 - Press “b” to start the ball bouncing
 - What happens! Nothing works!
- **Exercise 2:** Snap a pic.
- Your job: implement the three non-working classes
 - PongScore
 - PongPaddle
 - PongBall



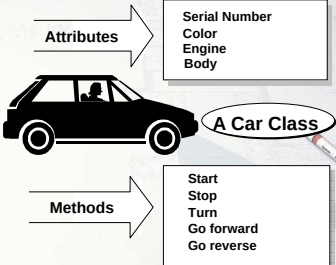
Review: What Are Classes?

- A **pattern** or **blueprint** that describes the common characteristics of a category of objects



The Class Definition Contains...

- Attributes
 - Object Data
- Methods
 - Object Actions



Class Definition

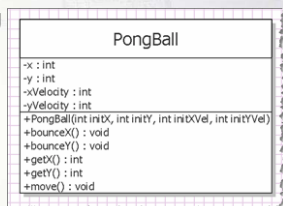
- When you **define a class**, you specify
 - The data attributes that hold its object's state
 - The methods that define object behavior
- Some methods modify the object
 - These are called **mutator** methods
 - **setBounds()** modifies a **Rectangle** object
- Some methods **return information about** an object
 - These kinds of methods are called **accessors**
 - The length of a **String**, width of a **Rectangle** object

The PongBall Class

- The **PongBall** class represents the ball in the game
 - The Pong program **uses** the **PongBall** class
- The Pong game first creates a **PongBall object**
 - It then calls the ball's **move()** method repeatedly
 - After calling **move()** it calls **getX()** and **getY()** to find out where the ball is located.
 - If it looks like the ball would "hit the wall", then the Pong program calls either **bounceX()** or **bounceY()** to cause the ball to rebound
- These are the methods that you have to write

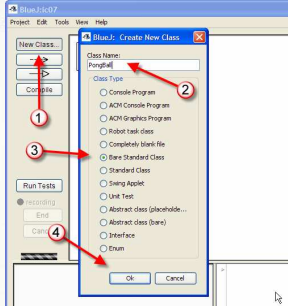
The UML Diagram

- Here's the UML class diagram for the **PongBall** class
 - **UML**: Unified Modeling Language: Standard OO diagramming tool (like blueprints in construction)
 - 4 data attributes
 - One constructor
 - Two accessors
 - Three mutators



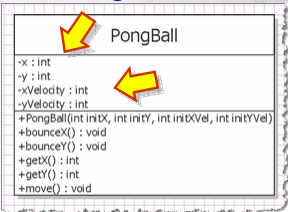
Implementation

- Close the Pong project and create an **ic05** project
- Create a new class
 - Click **New Class**
 - Name **PongBall**
Spelling important
 - Bare Standard Class**
 - Click **OK**
- Double-click to open
- Complete heading
- Exercise 3:** Snap a pic.



PongBall Data

- Four data attributes represent ball's **state**
 - x** and **y** represent the ball's position
 - xVelocity** and **yVelocity** represent the velocity of the ball in both the x and y directions
 - A negative **xVelocity** means ball is moving right to left



Defining Instance Variables

- We'll create **instance variables** for each attribute
- Instance variables** or **fields**
 - Defined **outside** of any method, but **inside class**
 - Can be used inside **any method** in the class

```
private type name [ = initial-value ];
```

- Modifier will **normally** be keyword **private**
 - Don't want variable used outside of class

Pong Instance Variables

- Add instance variables for each data attribute
 - Start with a JavaDoc comment

```

1 public class PongBall
2 {
3     /**
4      * Instance variables
5      */
6     private int x;
7     private int y;
8     private int xVelocity;
9     private int yVelocity;
10 }

```

Exercise 4
Snap a screenshot.

Mutator Methods

- Some methods **modify** the object
 - These are called **mutator** methods
- How do you tell if a method is a mutator?
- In **PongBall**, those are:
 - move()**, **bounceX()**, and **bounceY()**

```

PongBall
-x: int
-y: int
-xVelocity: int
-yVelocity: int
+PongBall(int initX, int initY, int initXVel, int initYVel)
+bounceX(): void
+bounceY(): void
+getX(): int
+getY(): int
+move(): void

```

Implementing Mutators

- Step 1:** Write the empty methods (public interface)
 - Access specifier:** **public** for methods
 - Return type:** mutator methods are **always void**
 - Names:** **move()**, **bounceX()**, and **bounceY()**
 - Arguments:** none for these, but still need **()**
- Place braces around empty bodies
- Exercise 5:** Write the method skeletons for these three mutators and snap a picture

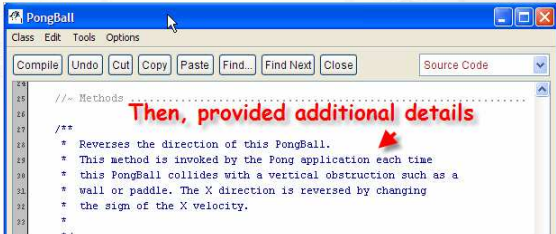
Commenting the Interface

- What actions should we perform in our methods?
 - Need to read specification supplied by the designer
 - Represent the **contract** that your class should fulfill
 - Provides information you'll need to implement behavior
- For each method, add documentation by adding special **JavaDoc** comments that describe:
 - A single sentence that describes what it does
 - What each of the parameters (if any) mean
 - What values the method returns

Writing

Exercise 6
Complete docs for bounceY, move
Snap a screenshot.

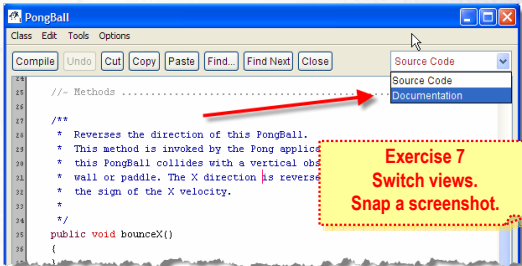
- Find the specification on the class links page
 - Locate the **PongBall bounceX()** method spec
- Add a **JavaDoc** comment to your code like this:



Then, provided additional details

Viewing the JavaDocs

- BlueJ let's you see your docs as the users will
 - In the editor, change Source Code to Documentation



Exercise 7
Switch views.
Snap a screenshot.

Modifying State

- For **bounceX()** and **bounceY()**, change the sign of the corresponding instance variable using assignment
- For **move()**, add the correct velocity value to the correct positioning element

```

31  * wall or paddle. The X direction is reversed by changing
32  * the sign of the X velocity.
33  */
34  public void bounceX()
35  {
36      xVelocity = -xVelocity;
37  }
  
```

Exercise 8
 Complete bounceX, move.
 Snap a screenshot.

Accessor Methods

- Some methods **return information about** an object
 - These kinds of methods are called **accessors**
 - Accessors are **value-producing** methods
- Value-producing methods:
 - Must have a return type **other than void**
 - Must end with a **return expression** (see page 97)

```

public returnType methodName( parameters )
{
    // statements
    return typeExpression;
}
  
```

PongBall Accessors

- PongBall** has two accessor methods, **getX()**, **getY()**
- Step 1:** Let's write the **JavaDoc** comments first
 - See the specification document for the info you need
 - Add an **@return** tag for every value-producing method

PongBall

```

-x : int
-y : int
-xVelocity : int
-yVelocity : int
+PongBall(int initX, int initY, int initXVel, int initYVel)
+bounceX() : void
+bounceY() : void
+getX() : int
+getY() : int
+move() : void
  
```

Accessor JavaDocs

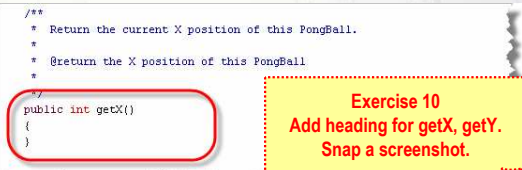
- Here's what the docs should look like for `getX()`
 - Pretty simple method. Most will have more info.



Exercise 9
Add JavaDoc for `getX`, `getY`.
Snap a screenshot.

Implementing Accessors

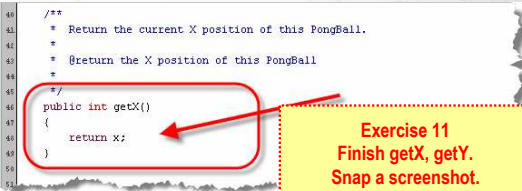
- Step 2:** Write the heading
 - Access specifier** : make these methods **public**
 - Return type** : accessors are **never** `void`
 - Names** : `getX()` and `getY()`
 - Arguments** : none for these methods, but need `()`



Exercise 10
Add heading for `getX`, `getY`.
Snap a screenshot.

Implementing Accessors

- Step 3:** Write the code in the body
 - Remember, every accessor **must** end with a return



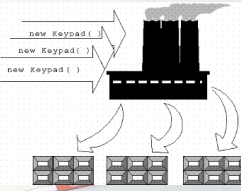
Exercise 11
Finish `getX`, `getY`.
Snap a screenshot.

Initializing PongBall

- Instance variables are **automatically initialized**
 - Given values **0**, **false**, or **null** depending on type
- Normally want to **explicitly** initialize your objects
 - Pong expects to place ball in center of the screen
 - Note that there are **no mutator** methods that allow a **client** (the Pong class) to set the values for x and y
- This is done by writing a **constructor**

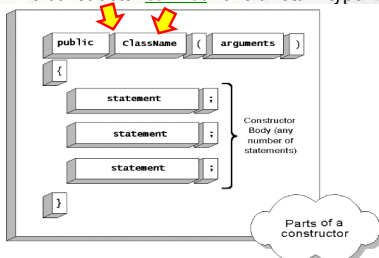
What Are Constructors?

- Constructors are defined inside a class like a method
 - But, they are **not called directly** like a method
 - Instead, called when you use the **new** operator
 - Java uses the constructor to initialize your object
- A constructor is like a factory that **creates** an object. It is responsible for initializing an object's fields when the object is created.



Constructor Syntax

- Constructors look like methods, with two constraints:
 - The name is **always** the same as the class
 - The constructor **cannot** have a return type



Constructor Interface

- **Step 1:** Write the header
 - **Access specifier** : is always **public**
 - **Return type** : constructors **never** have a return type
 - **Names** : the same as the class: **PongBall**
 - **Parameters** : (or arguments) one for each initial value
 - Supply both the type and a name for each variable
 - Separate each declaration by commas

```
20 //~ Constructor .....
21
22 public PongBall(int initX, int initY,
23               int initXVelocity, int initYVelocity)
24 {
25 }
26
```

What are Parameters?

- **Most** methods/constructors require extra information
 - **Math.sqrt()** requires the number to operator on
 - **PongBall** needs to know initial values for fields
- Sent to your method via **"parameters"** or **"arguments"**
- There are two kinds of parameters:
 - **Formal parameters** are variables used in the method
 - Designed specifically to for passing information
 - **Actual parameters** or **arguments** are the values that are supplied when the method is called

Local Variables

- A **formal parameter** is **like** a local variable, but designed especially to hold method-input information
- Here's how a regular local variable is defined:

```
public void aMethod()
{
    int aLocalVar = 32;
}
```

- The variable **aLocalVar** is an "int-shaped bucket" holding an **int** value, initialized using **assignment**
- Has exactly same value each time method is called

Formal Parameters

- Formal parameter is similar to a local variable, **except**:
 - It is declared **inside** the argument list
 - You must include a type for **each** argument
 - It is given a value when the method is called
- Here is **aMethod()** with a formal argument

```
public void aMethod(int arg)
{
    int aLocalVar = 32;
}
```

Constructor JavaDoc

- Step 2:** Document the constructor
 - Use one **@param** tag for each parameter
 - Separate name with space, end with a period

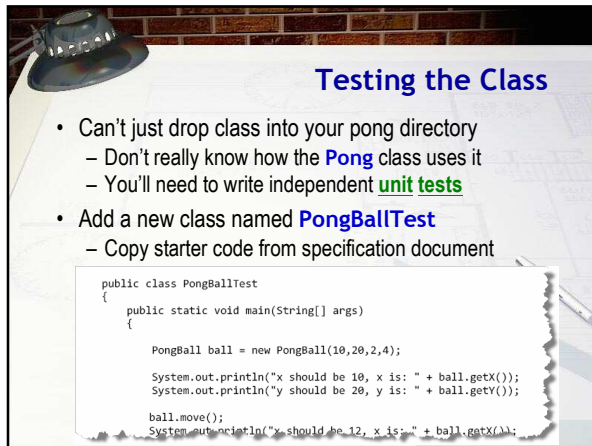
```
21 /**
22  * Construct a new PongBall with a specified initial position and velocity.
23  *
24  * @param initX the initial X position of the new PongBall.
25  * @param initY the initial Y position of the new PongBall.
26  * @param initXVelocity the initial velocity of the new PongBall
27  *   in the X direction.
28  * @param initYVelocity the initial velocity of the new PongBall
29  *   in the Y direction.
30  */
31 public PongBall(int initX, int initY,
32                 int initXVelocity, int initYVelocity)
33 {
34 }
35
36
```

Exercise 12
Complete constructor JavaDoc.
Snap a screenshot of documentation view.

Constructor Implementation

- Step 3:** Initialize **each** of the instance variables
 - Assign the actual parameters to the fields
 - Check carefully; make sure not to accidentally reverse

```
29 * @param initYVelocity the initial velocity of the new PongBall
30 *   in the Y direction.
31 */
32 public PongBall(int initX, int initY,
33                 int initXVelocity, int initYVelocity)
34 {
35     x = initX;
36     y = initY;
37     xVelocity = initXVelocity;
38     yVelocity = initYVelocity;
39 }
40
```



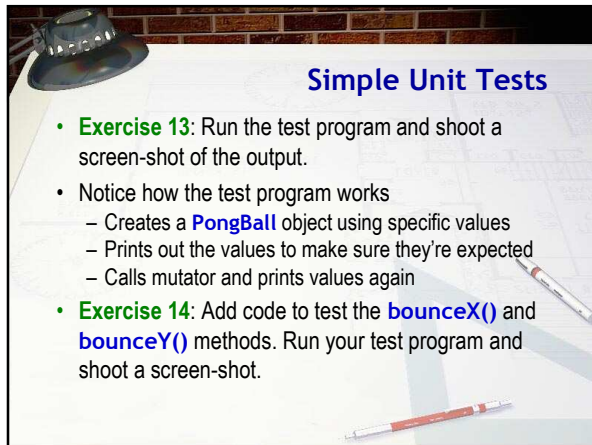
Testing the Class

- Can't just drop class into your pong directory
 - Don't really know how the **Pong** class uses it
 - You'll need to write independent **unit tests**
- Add a new class named **PongBallTest**
 - Copy starter code from specification document

```
public class PongBallTest
{
    public static void main(String[] args)
    {
        PongBall ball = new PongBall(10,20,2,4);

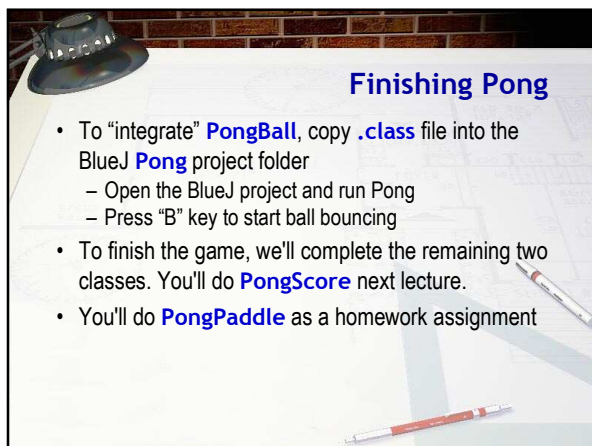
        System.out.println("x should be 10, x is: " + ball.getX());
        System.out.println("y should be 20, y is: " + ball.getY());

        ball.move();
        System.out.println("x should be 12, x is: " + ball.getX());
    }
}
```



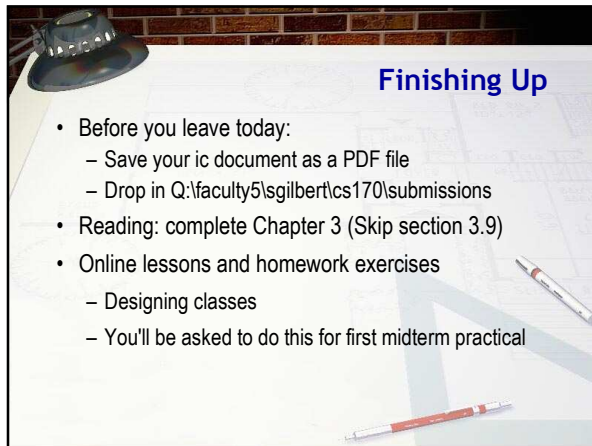
Simple Unit Tests

- **Exercise 13:** Run the test program and shoot a screen-shot of the output.
- Notice how the test program works
 - Creates a **PongBall** object using specific values
 - Prints out the values to make sure they're expected
 - Calls mutator and prints values again
- **Exercise 14:** Add code to test the **bounceX()** and **bounceY()** methods. Run your test program and shoot a screen-shot.



Finishing Pong

- To "integrate" **PongBall**, copy **.class** file into the BlueJ **Pong** project folder
 - Open the BlueJ project and run Pong
 - Press "B" key to start ball bouncing
- To finish the game, we'll complete the remaining two classes. You'll do **PongScore** next lecture.
- You'll do **PongPaddle** as a homework assignment



Finishing Up

- Before you leave today:
 - Save your ic document as a PDF file
 - Drop in Q:\faculty5\sgilbert\cs170\submissions
- Reading: complete Chapter 3 (Skip section 3.9)
- Online lessons and homework exercises
 - Designing classes
 - You'll be asked to do this for first midterm practical
